UNIVERSITY OF WAH(UOW)

DEPARTMENT OF COMPUTER SCIENCE

Lab Task # 06

COURSE: Data Structure & Algorithm Lab_201L

SUBMITTED TO: Ms.Muniba Khan

SUBMITTED BY: Muhammad Talha_CS.3ndB

Registration NO: UW-24-CS-BS-101

Task # 01

```cpp
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

void insert_at_beg(Node *&head)
{
    int value;
    cout << "Enter Value to enter: ";
    cin >> value;

    Node *newNode = new Node();
    newNode->data = value;
    newNode->next = head;
    head = newNode;

    cout << "Successfully Added." << endl;
}
```

```cpp
void insert_at_end(Node *&head)
{
    int value;
    cout << "Enter Value to add: ";
    cin >> value;

    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
    {
        head = newNode;
        cout << "Successfully Added at end." << endl;
        return;
    }

    Node* temp = head;
    while (temp->next != NULL)
    {
        temp = temp->next;
    }
    temp->next = newNode;

    cout << "Successfully Added at end." << endl;
}
```

```
void insert_at_pos(Node *&head)
{
    int pos, value;
    cout << "Enter Position: ";
    cin >> pos;

    cout << "Enter Value to Add: ";
    cin >> value;

    if (pos < 1)
    {
        cout << "Invalid Position!" << endl;
        return;
    }
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = NULL;
    if (pos == 1)
    {
        newNode->next = head;
        head = newNode;
        cout << "Successfully Added." << endl;
        return;
    }
    Node* temp = head;
    for (int i = 1; i < pos - 1; i++)
    {
        if (temp == NULL)
        {
            cout << "Position out of range!" << endl;
```

```
            temp = temp->next;
        }
        if (temp == NULL)
        {
            cout << "Position out of range!" << endl;
            return;
        }

        newNode->next = temp->next;
        temp->next = newNode;

        cout << "Successfully Added at Position " << pos << endl;
}

void display(Node *head)
{
    Node* temp = head;

    if (head == NULL)
    {
        cout << "List is empty!" << endl;
        return;
    }
```

```
    while (temp != NULL)
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL";
}
int main()
{
    int choice = 0;
    Node *head = NULL;

    while (choice != 5)
    {
        cout << "\n== Linked List ==" << endl;
        cout << "1: Add at End" << endl;
        cout << "2: Add at Beginning" << endl;
        cout << "3: Add at Position" << endl;
        cout << "4: Display All Elements" << endl;
        cout << "5: Exit" << endl;
        cout << "Your Choice: ";
        cin >> choice;

        switch (choice)
        {
        case 1:
            insert_at_end(head);
            break;
```

```
        case 2:
            insert_at_beg(head);
            break;

        case 3:
            insert_at_pos(head);
            break;

        case 4:
            display(head);
            break;

        case 5:
            cout << "Exiting..." << endl;
            break;

        default:
            cout << "Wrong Choice!" << endl;
        }
    }
```

Output

```
== Linked List ==
1: Add at End
2: Add at Beginning
3: Add at Position
4: Display All Elements
5: Exit
Your Choice: 4
10 -> 15 -> 20 -> NULL
```

Task # 02

Deletion

```
void searchElement(Node* head, int value)
{
    int pos = 1;
    Node* temp = head;

    while(temp != NULL)
    {
        if(temp->data == value)
        {
            cout << "Element " << value << " found at position " << pos << endl;
            return;
        }
        temp = temp->next;
        pos++;
    }

    cout << "Element " << value << " not found in the list." << endl;
}
```

Searching

```
void deletePos(Node*& head, int pos)
{
    if(head == NULL || pos < 1)
    {
        cout << "Invalid Position!" << endl;
        return;
    }

    if(pos == 1)
    {
        Node* temp = head;
        head = head->next;
        delete temp;
        cout << "Node deleted at position 1." << endl;
        return;
    }
```

```
    Node* temp = head;
    for(int i = 1; i < pos - 1; i++)
    {
        if(temp == NULL || temp->next == NULL)
        {
            cout << "Position out of range!" << endl;
            return;
        }
        temp = temp->next;
    }

    Node* delNode = temp->next;

    if(delNode == NULL)
    {
        cout << "Position out of range!" << endl;
        return;
    }

    temp->next = delNode->next;
    delete delNode;

    cout << "Node deleted at position " << pos << endl;
}
```

```
while (choice != 7)
{
    cout << "\n== Linked List Menu ==" << endl;
    cout << "1: Add at End" << endl;
    cout << "2: Add at Beginning" << endl;
    cout << "3: Add at Position" << endl;
    cout << "4: Display List" << endl;
    cout << "5: Search Element" << endl;
    cout << "6: Delete at Position" << endl;
    cout << "7: Exit" << endl;
    cout << "Your Choice: ";
    cin >> choice;
```

```
case 5: {
    int value;
    cout << "Enter value to search: ";
    cin >> value;
    searchElement(head, value);
    break;
}
```

```
case 6: {
    int pos;
    cout << "Enter position to delete: ";
    cin >> pos;
    deletePos(head, pos);
```

Output

```
== Linked List Menu ==
1: Add at End
2: Add at Beginning
3: Add at Position
4: Display List
5: Search Element
6: Delete at Position
7: Exit
Your Choice: 5
Enter value to search: 15
Element 15 found at position 3
```

```
== Linked List Menu ==
1: Add at End
2: Add at Beginning
3: Add at Position
4: Display List
5: Search Element
6: Delete at Position
7: Exit
Your Choice: 6
Enter position to delete: 2
Node deleted at position 2
```

```
== Linked List Menu ==
1: Add at End
2: Add at Beginning
3: Add at Position
4: Display List
5: Search Element
6: Delete at Position
7: Exit
Your Choice: 4
5 -> 15 -> 20 -> NULL
```